# Experiment 6

**Student Name: Prabhakar kr. Jha** **UID: 23BCS12284**
**Branch: CSE** **Section/Group: KRG 3-B**
**Semester: 6th** **Date of Performance:25/02/2026**
**Subject Name: System Design** **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable video-sharing and streaming platform similar to YouTube that supports video uploads, adaptive streaming (HLS/DASH), likes, comments, and massive concurrent users with high availability and low latency.

2. **Objective**:
   - To identify functional and non-functional requirements of a YouTube-like system
   - To design High-Level Architecture (HLD) for a video-sharing platform
   - To understand Manifest files and Adaptive Bitrate Streaming
   - To implement Like and Comment handling in distributed systems
   - To understand the role of CDN, Kafka, Cache, and Object Storage
   - To apply scalability and CAP theorem concepts

3. **Tools:**

- **Draw.io** – System Architecture & HLD Diagram
- **MySQL** – User, subscription, and payment data
- **MongoDB / NoSQL DB** – Video metadata storage
- **ElasticSearch** – Search functionality for movies and TV shows
- **Apache Kafka** – Asynchronous event streaming and video pipeline coordination
- **Redis / CDN Cache** – Caching frequently accessed content
- **Blob Storage (Amazon S3 equivalent)** – Video and media storage

4. **System Requirements:**

### A. Functional Requirements:-

1. User can register and login
2. User can upload videos
3. System transcodes videos into multiple resolutions (360p, 720p, 1080p, 4K)
4. System generates HLS (.m3u8) or DASH (.mpd) manifest files
5. User can search videos

6. User can watch videos with adaptive streaming
7. User can like/unlike videos
8. User can comment and reply on videos
9. System displays real-time like count
10. System supports pagination of comments

### B. Non-Functional Requirements:-

1. **Scalability**

- Target Users: 500M+
- Millions of concurrent video streams
- High write traffic for likes

2. **Availability vs Consistency (CAP Theorem)**

- Availability >>> Consistency

**Reason**:

- Video playback must not stop
- Slight delay in like count update is acceptable
- Comments may be eventually consistent

**Example**:

- Video not loading → Critical
- Like count delayed by few seconds → Acceptable

3. **Latency**

- Video start latency < 2 seconds
- API latency 50–100 ms
- Video should play with zero or negligible buffering
-

## 5. High Level Design (HLD):

The system follows a microservices-based architecture:

**API Gateway**

- Handles authentication, authorization, routing, and rate limiting

**User Service**

- User registration, login, JWT-based authentication
- Stores user data in MySQL

**Subscription & Payment Service**

- Manages subscription plans and payments
- Ensures strong consistency for transactions

**Search Service**

- Uses ElasticSearch for fast movie/TV show search

**Video Metadata Service**

- Stores metadata (title, description, thumbnails) in NoSQL DB

**Video Upload & Processing Pipeline**

- Uploaded videos are stored in Blob Storage
- Chunker Service splits videos into small segments
- Video Encoder generates multiple resolutions (480p, 720p, 1080p, 4K)
- Kafka coordinates asynchronous processing

**Streaming Service**

- Generates HLS (.m3u8) or DASH (.mpd) manifests
- Client fetches chunks based on bandwidth

**CDN**

- Caches most frequently watched video chunks
- Reduces latency and server load

## 6. **Video Streaming Workflow**:

- User clicks Play Video

- Client requests manifest file (.m3u8 / .mpd)
- Manifest contains URLs of video chunks with different bitrates
- Client measures network bandwidth
- Appropriate resolution chunks are fetched dynamically
- CDN serves cached chunks if available
- Video playback continues smoothly with adaptive bitrate

## 7. Scalability Solution:

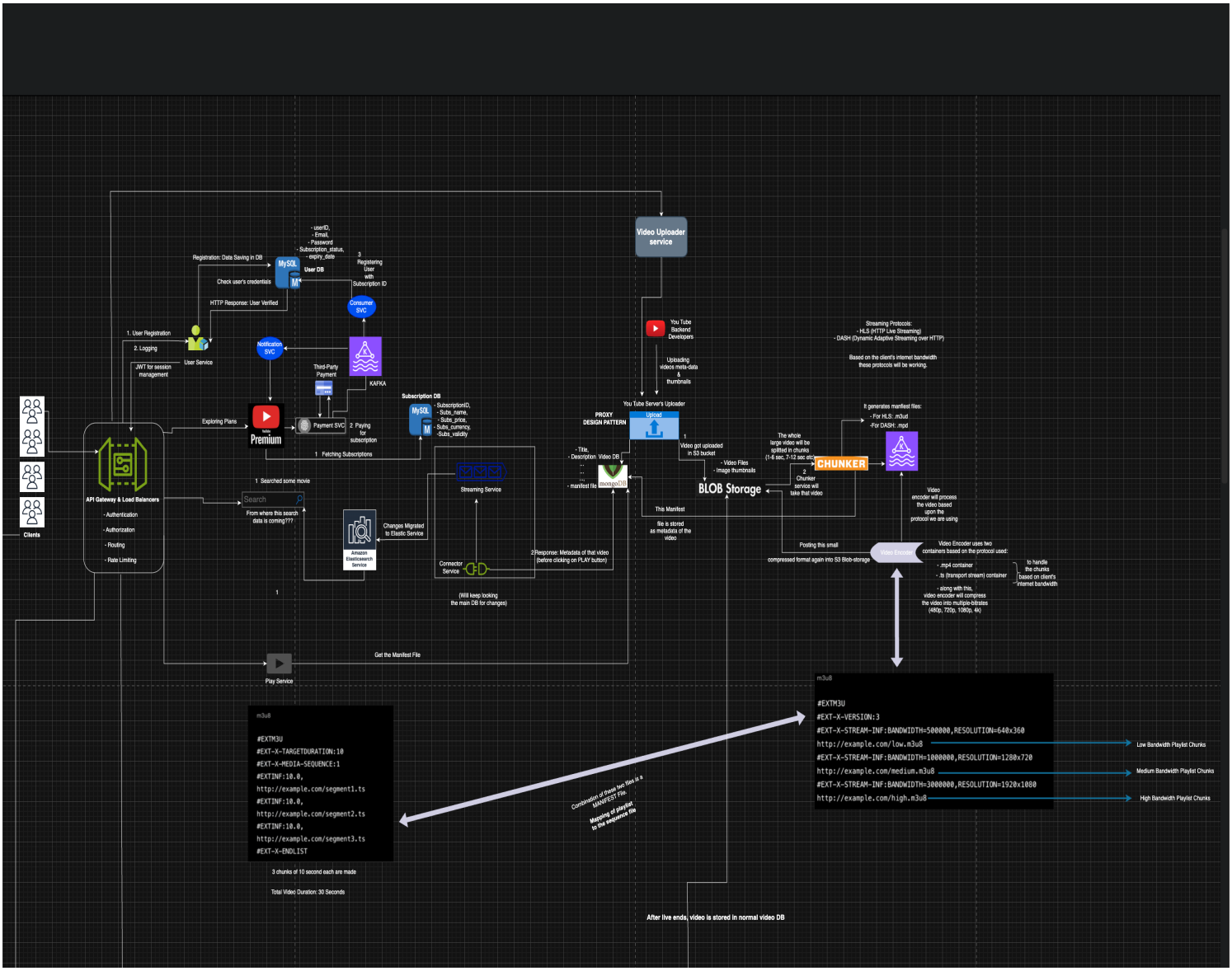
- Horizontal scaling of all microservices
- API Gateway acts as load balancer
- Kafka decouples video processing pipeline
- CDN caching minimizes latency and origin load
- Blob storage efficiently stores large video files
- Stateless services allow easy scaling

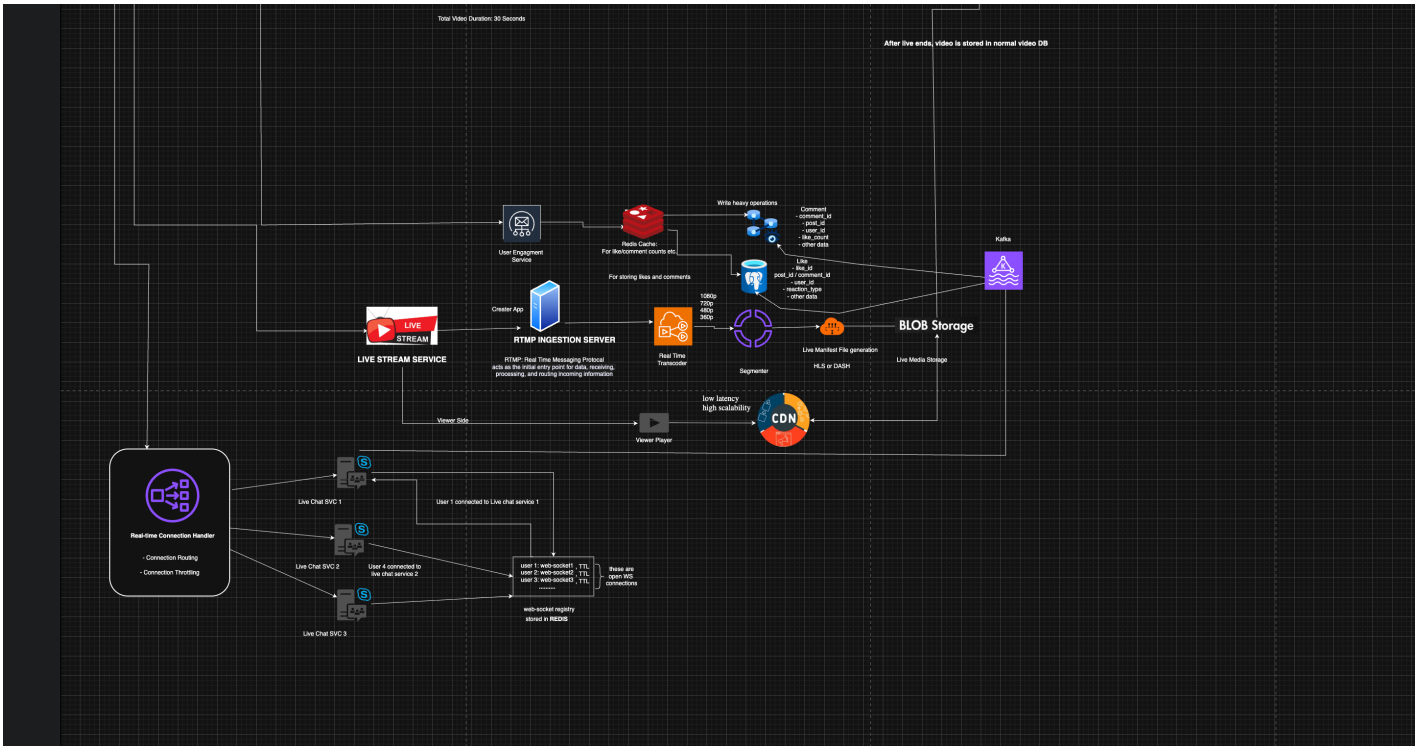
## 8. Like Service

- User can like/unlike video
- Prevent duplicate likes (unique user_id + video_id)
- Redis stores real-time like counter
- Async update to Likes DB
    - Flow:
- User → API Gateway → Like Service → Redis (INCR)
  Redis → Background Worker → Likes DB
- Reason for Cache:
  High write traffic must not overload database

## 9. Comment Service

- Add comment
- Reply to comment
- Delete comment
- Pagination support
- Sort by latest/top






## 7. Learning Outcomes (What I Have Learnt):

- Understood how real-world OTT platforms like Netflix are designed
- Learned difference between functional and non-functional requirements
- Gained strong understanding of adaptive bitrate streaming (HLS/DASH)
- Understood CAP theorem trade-offs in streaming systems
- Learned how Kafka, CDN, and chunking pipelines work together
- Learned how to design highly available, low-latency distributed systems